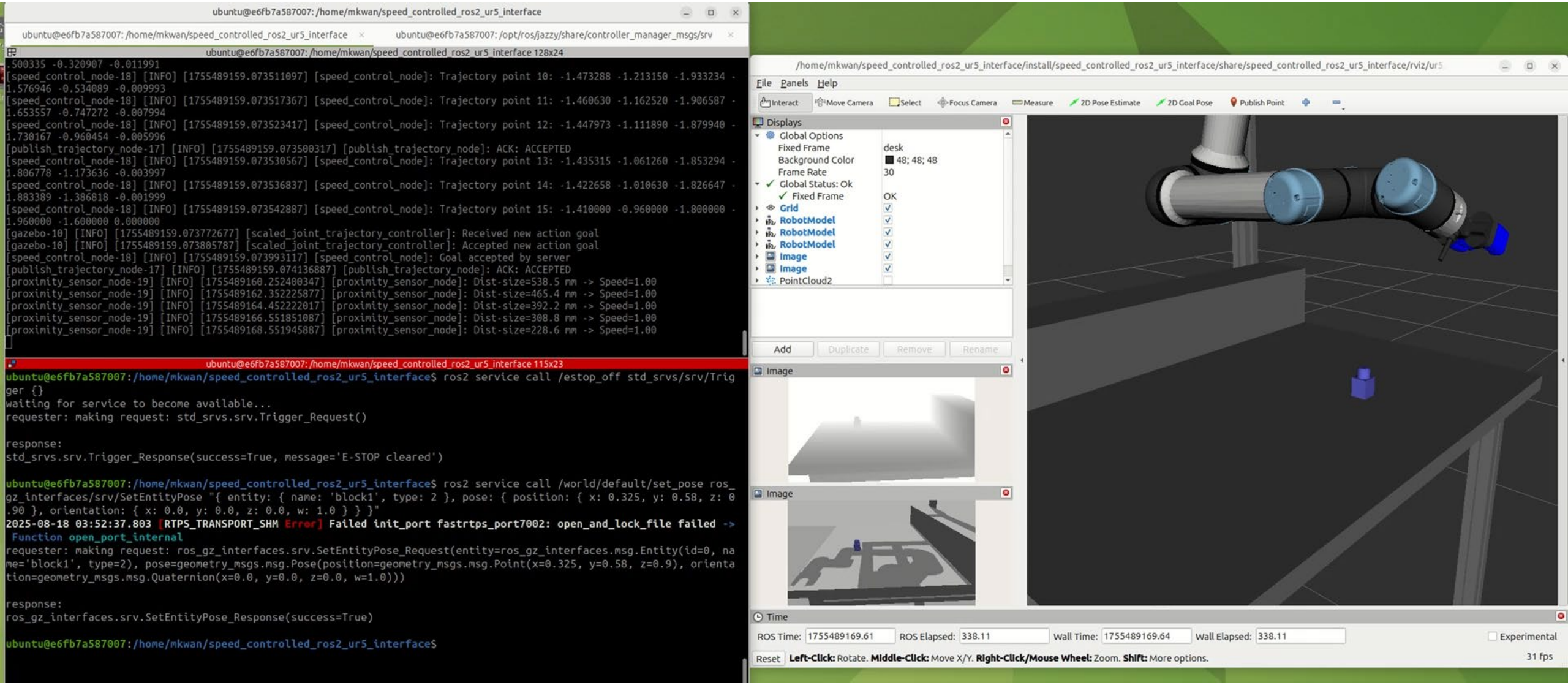
Speed Controlled Cobot with E-Stop

Presented by
Matt Kwan

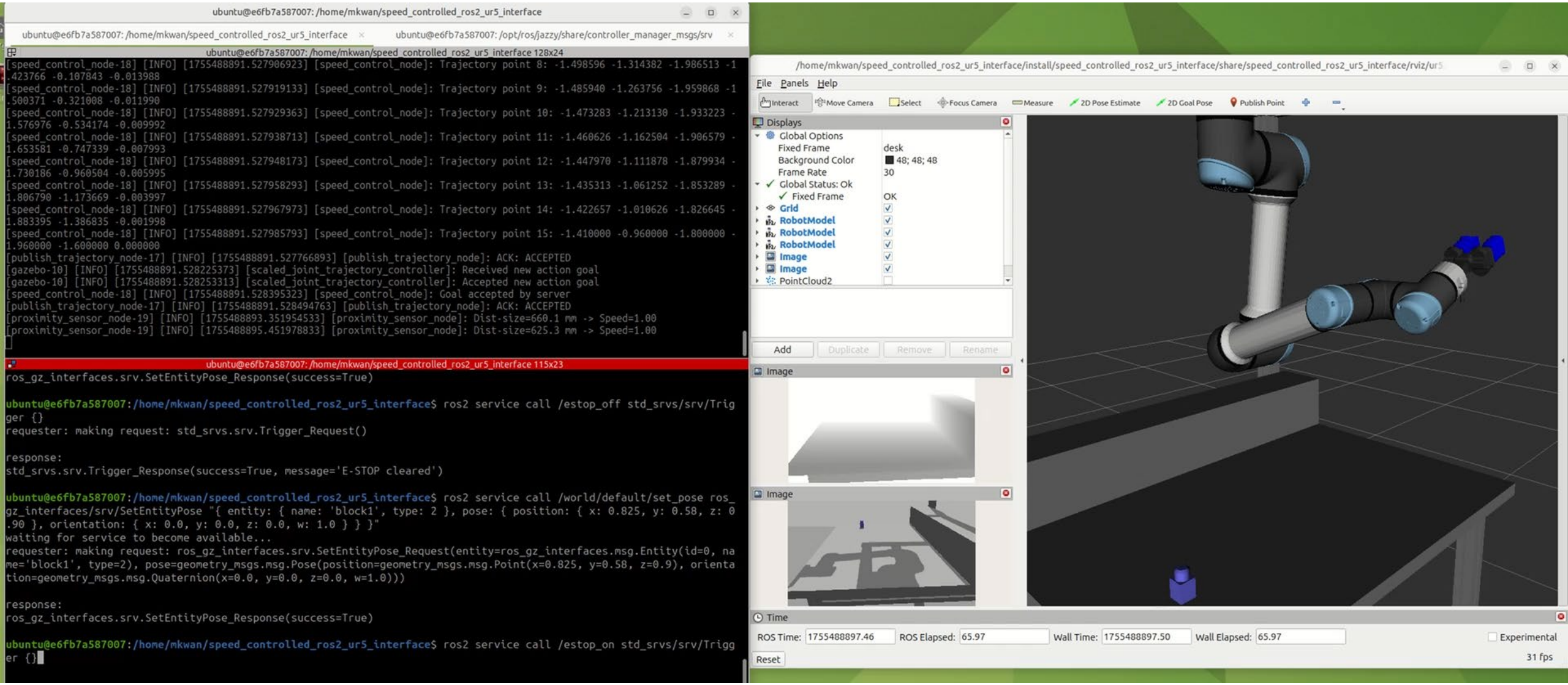
Presented for
Analog Garage
August 2025

Introduction & Approach

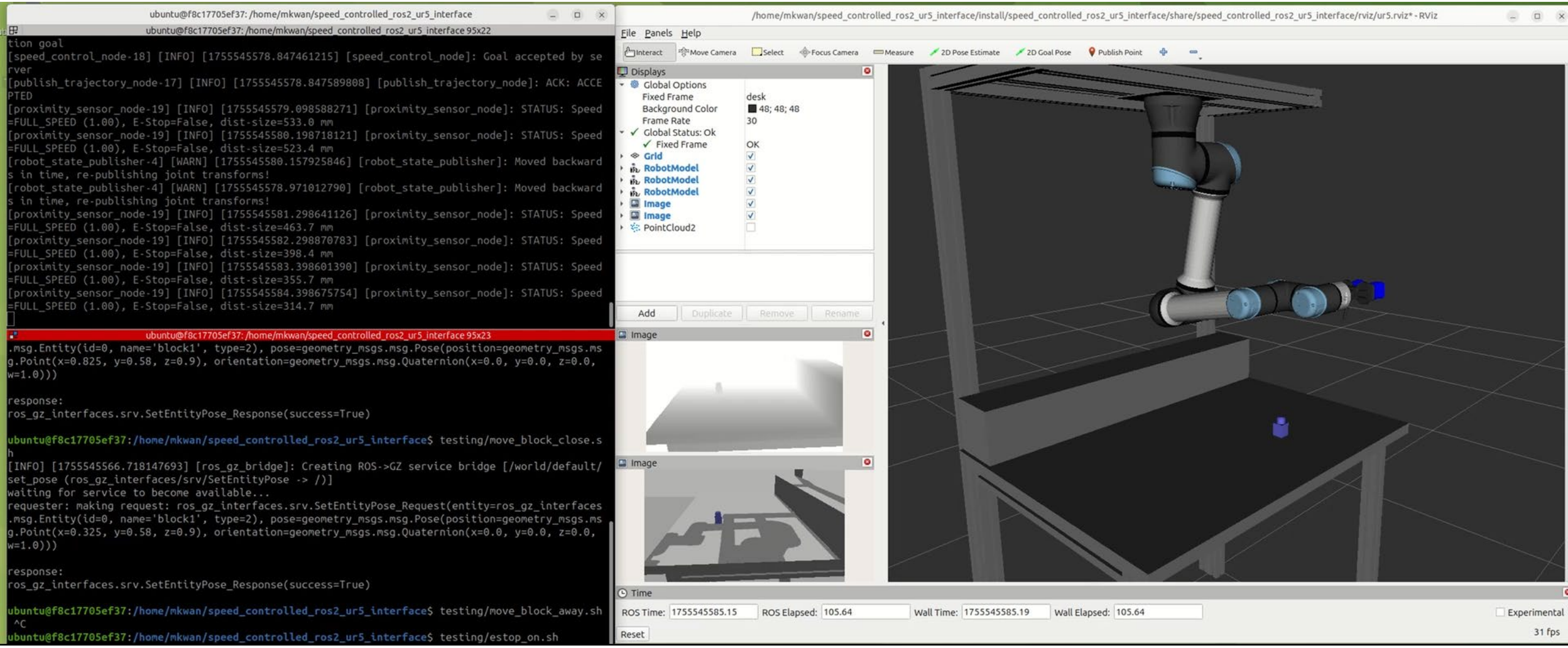
- Objective:
  - Design and implement a modular control system that adjusts the cobot's speed based on proximity input and responds immediately to an emergency stop signal.
- Tech stack:
  - C++, Python, ROS2
  - Forked from UR5 Simulation
- Components:
  - Proximity Sensor Node (Python)
  - E-Stop Controller Node (Python)
  - Speed Control Node (C++)
  - Publish Trajectory Node (C++, from UR5)
- Key features:
  - 3 speed states: FULL_SPEED, SLOW, STOP
  - E-Stop toggle with saved state
  - Automatic proximity detection with simulation object
  - Proximity hysteresis via window averaging
- Next: Results & Demonstration



Proximity sensor and speed control demonstration. The cobot slows then stops when approaching the block. When the block is moved, the cobot no longer detects its proximity and resumes full-speed operation. Shown twice.



E-Stop demonstration. A ROS2 service activates the E-Stop and another ROS2 service deactivates the E-Stop. When the E-Stop is activated, the cobot freezes in place. When the E-Stop is cleared, operation resumes normally.



Full scenario demonstration. (1) Proximity sensing triggers a speed change. (2) E-Stop interrupting operation during proximity detection. (3) Once both the obstacle and the E-Stop are cleared, normal operation resumes. Shown twice.

Discussion & Conclusion

- Key decisions:
  - Integrating with UR5 Simulator has significant development cost but is more representative and would likely transfer decently onto real UR5 hardware
  - ROS2's controllers mandate non-blocking concurrency which allow better scaling and elasticity for more complex processing and/or constrained hardware
  - The E-Stop does not simply command a 0.0 speed; it disconnects the entire joint trajectory controller to further ensure motion commands will fail, in addition to setting a 0.0 speed and toggling an E-Stopped state across the platform
  - Using a circular buffer for hysteresis limits noisy proximity data
- Future work:
  - Realistic object detection via LiDAR or stereo vision point cloud
    - Easy incremental change would be to add random noise to the proximity information
  - Modular unit testing with deterministic output for CI/CD
  - Tuning and stress-testing speed limits for cobot via points per trajectory, base time per point, joint and time tolerances, communication limits
  - Improve exception handling, error state handling, edge cases, complex and/or highly specific trajectories

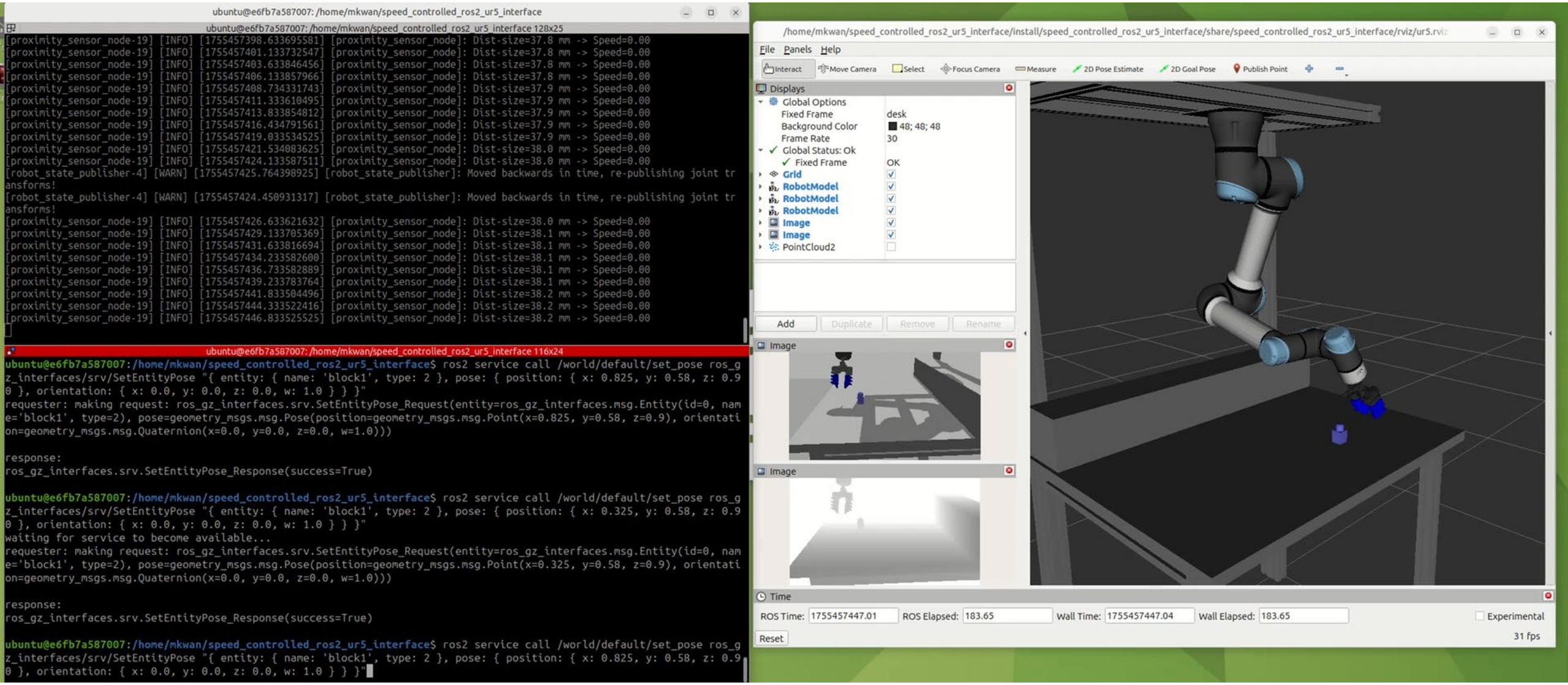
Thank you!

Presented by: Matt Kwan

LinkedIn

Link to Github repo

Appendix



Similar demonstration of the proximity sensor integration. Zoomed-out perspective.